Journal of Signal Processing Systems (2026) 98:10
https://doi.org/10.1007/s11265-025-01978-3

RESEARCH

Dynamic Consensus Communication Mechanism for Large Language Model-Based Multi-Agent Systems

Liancheng Yang[1] · Sining Li[2] · Aobo Deng[2]

Received: 22 July 2025 / Revised: 15 September 2025 / Accepted: 20 September 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract
Large Language Models (LLMs) often struggle with multi-step reasoning tasks. To address this, LLM-based Multi-Agent Systems (LLM-MAS) have been introduced, where agents interact to reach consensus. However, these systems face two major issues: hallucination propagation, where incorrect responses mislead others, and communication redundancy, where excessive interactions increase token cost without improving performance. This paper proposes a unified solution: Consensus-driven Community Agent Communication (AC^3). Each agent's response is decomposed into an answer and an analysis, allowing for precise tracking of misleading content and redundant patterns. Agents are clustered into communities based on answers, and selected representatives—chosen by analysis quality—conduct cross-community discussion. A confirmation mechanism finalizes the consensus. Experiments on multiple datasets show that AC^3 significantly reduces hallucinations and communication overhead while maintaining or improving reasoning accuracy. These results demonstrate the effectiveness and scalability of AC^3 in enhancing LLM-based multi-agent systems.

Keywords Large language model · Multi-agent system · Communication topology · Communication redundancy · Hallucination propagation

1 Introduction

With the rapid development of artificial intelligence and big data technologies, ensuring the security and reliability of large-scale systems has become a critical challenge. Big data security encompasses multiple dimensions, including data integrity, privacy protection, and maintaining efficient communication under resource constraints, which are essential for distributed environments. Prior research provides valuable insights into these areas: studies on system usability and human–machine interaction [1] inform user-centric secure system design, while work on dynamic resource scheduling and energy-aware computing [2, 3] emphasizes strategies for real-time efficiency under limited resources. Furthermore, research on communication-efficient algorithms for large-scale optimization [4] highlights the need to minimize transmission overhead without compromising robustness. These findings collectively indicate that achieving both efficiency and reliability requires adaptive architectures and intelligent communication strategies. Such concerns are increasingly relevant to collaborative reasoning systems, where information consistency and computational cost must be jointly optimized.

Large Language Models (LLMs) have achieved remarkable success in natural language understanding and generation, excelling in tasks such as question answering and reasoning. Representative models like GPT [5], LLaMA [6], and DeepSeek [7] demonstrate strong generalization by leveraging massive textual corpora. However, single LLMs often struggle with multi-step logical reasoning, such as mathematical problem solving or commonsense inference, because these tasks require consistent deduction across multiple stages—beyond the implicit reasoning capacity of isolated models.

✉ Sining Li
2025282120187@whu.edu.cn

Liancheng Yang
yanglc21@chinaunicom.cn

Aobo Deng
2023282120174@whu.edu.cn

1 China United Network Communications Co., Ltd., Guangzhou Branch, Guangzhou, China

2 Wuhan University, Electronic Information School, Wuhan, China

Published online: 14 January 2026

To overcome these limitations, techniques like Chain-of-Thought prompting and Self-Consistency decoding have been proposed. While effective in some settings, these methods depend on handcrafted templates and lack adaptability to dynamic or unseen contexts. This has motivated a shift toward LLM-based Multi-Agent Systems (LLM-MAS), where multiple agents collaborate through iterative discussions to critique, refine, and converge on solutions. This paradigm, inspired by human deliberation, has shown promise in improving reasoning accuracy and interpretability.

Despite this progress, LLM-MAS introduces two tightly coupled challenges that hinder practical deployment. 1) hallucination propagation: erroneous information generated by one agent can spread through the network, amplifying local mistakes into systemic failures—similar to data integrity risks in distributed systems. 2)communication redundancy: repetitive or low-value exchanges inflate token consumption and increase latency without contributing new insights. These problems reinforce each other: redundant interactions amplify the spread of incorrect information, while hallucinated content triggers further unnecessary communication (Fig. 1).

Existing approaches often tackle these problems in isolation—suppressing hallucinations via verification mechanisms or reducing redundancy through static topologies—but lack a unified and adaptive strategy to mitigate both issues in dynamic, multi-round settings.

To address this gap, we propose AC^3 (Consensus-driven Community Agent Communication), a unified communication mechanism for LLM-MAS. The key idea is to decompose each agent's response into two components—*answer* and *analysis*—enabling fine-grained tracking of information flow. Building on this structure, AC^3 integrates three modules: (i) *community formation* based on answer similarity, (ii) *representative selection* using analysis quality, and (iii) a *confirmation mechanism* with referee agents to validate consensus outcomes.

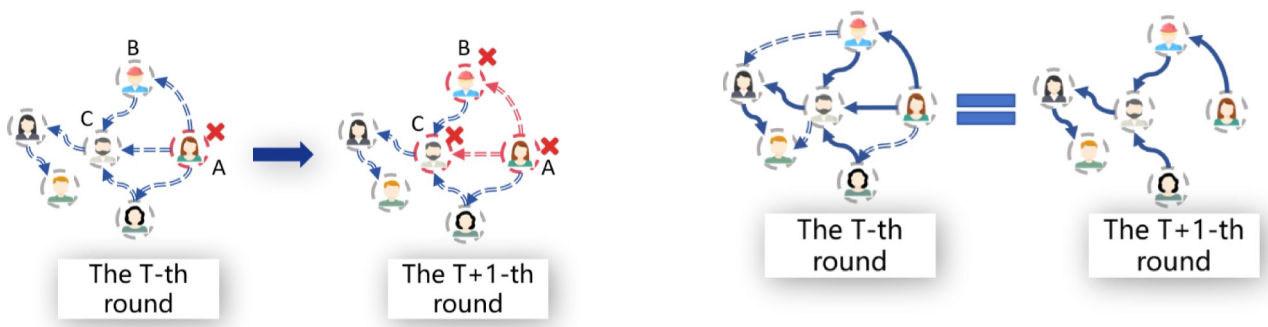
We evaluate AC^3 across multiple reasoning benchmarks and compare it against state-of-the-art multi-agent communication strategies. Experimental results show that AC^3 not only reduces hallucination propagation and token-level overhead but also maintains or improves reasoning accuracy as the number of agents increases.

The main contributions of this paper are as follows:

1. We propose the first unified approach that simultaneously mitigates hallucination propagation and communication redundancy in LLM-based multi-agent systems.
2. We introduce AC^3, a dynamic, content-aware communication mechanism that combines community clustering, representative selection, and consensus validation into a single adaptive workflow.
3. Extensive experiments on seven reasoning benchmarks (e.g., GSM8K, MathQA, CommonsenseQA) demonstrate that AC^3 reduces token usage by up to 83% while maintaining or improving reasoning accuracy compared to strong baselines.

2 Related Work

To better contextualize our AC^3 framework, we first review existing research relevant to LLM-MAS—focusing on three key areas that align with the critical challenges of LLM-MAS: the development of LLM-MAS



(a) Hallucination propagation: an incorrect answer from Agent A misleads Agents B and C, causing systemic error in the next round.

(b) Communication redundancy: solid lines denote effective reasoning paths, while dashed lines indicate redundant transmissions that do not alter final results.

Fig. 1 Two key challenges in LLM-MAS: hallucination propagation and communication redundancy, which jointly degrade reasoning efficiency and reliability.

architectures, techniques to mitigate hallucination propagation, and strategies to reduce communication redundancy. This review helps clarify where our work fits within current research.

2.1 Large Language Model-Based Multi-Agent Systems

The foundation of modern LLM-MAS can be traced to OpenAI's 2018 work on AI Safety via Debate [8, 9], which introduced adversarial debates between language models to promote interpretability. Follow-up research expanded on this idea using communication protocols like role-playing [10] and dynamic agent networks [11].

In 2023, structured multi-agent frameworks began to emerge. AutoAgents [12] proposed role-based workflows for task decomposition, while theoretical works identified risks such as the Role-Playing Paradox [13] and Degeneration of Thought [14], emphasizing the importance of inter-agent deliberation.

Practical implementations span various applications: Gorilla [15] supports tool-integrated decision-making, AgentSims [16] enables multi-agent simulations in strategic environments, and Reflexion [17] incorporates closed-loop feedback for continual learning. NetSafe [18] further improves system resilience by introducing fault-tolerant communication structures.

These developments reveal two dominant trends: structured communication protocols such as role-based workflows and scripted conversations, and resilience-oriented frameworks that introduce reflection, debate, or error correction for robustness.

However, few works explore how error propagation and inefficient communication interact—especially under dynamic, multi-round collaboration. Recent advances in dynamic agent networks emphasize adaptive team configuration. The Dylan framework [19] uses a two-stage paradigm for LLM-agent collaboration, selecting agents via "importance scores" to form optimal teams and performing well in code generation/reasoning. However, it only optimizes team size—not semantic-based agent clustering—leaving it vulnerable to hallucination propagation.

Another key work is AgentSGEN [20], which leverages evaluator-editor iterative collaboration for semantic consistency in synthetic data generation, with role separation enabling fine-grained control. Yet it lacks reasoning quality-based dynamic topology adjustment—a core capability of AC^3. These gaps highlight the need for semantic analysis in topology optimization, a key innovation of AC^3.

2.2 Hallucination Propagation in LLM-MAS

Hallucination propagation is a major challenge in multi-agent environments, where incorrect reasoning from one agent can spread across the network, misleading others and undermining consensus.

The Markov Debate framework [21] treats agent interactions as state transitions in a Markov chain, allowing for the tracking of error paths. While conceptually powerful, this approach is computationally expensive in large-scale settings.

A three-tier agent architecture [22] assigns generation, interrogation, and revision roles to improve consistency. However, semantic drift between tiers may lead to false positives or missed errors.

Other approaches employ external arbiters [23] to re-rank conflicting responses based on confidence scores, or leverage self-checking agents like SelfCheckAgent [24], which apply rule-based consistency checks. These mechanisms reduce hallucinations but often incur significant token cost and struggle with subtle logical errors.

Overall, hallucination mitigation techniques face a trade-off between accuracy and scalability. Most existing methods either require additional computational resources or fail to generalize across diverse tasks and reasoning trajectories.

2.3 Communication Redundancy and Topological Optimization

Communication redundancy arises when agents repeatedly exchange low-value or duplicate messages, leading to excessive token usage without improving inference quality. To address this, researchers have explored both static and dynamic topological structures.

Static topologies constrain agent interactions to fixed graphs (e.g., star, tree, ring). Li et al. [25] demonstrate that reducing connection density in small agent networks reduces token cost without sacrificing performance. However, static topologies lack adaptability to changing task contexts or agent behavior.

Dynamic topologies enable agents to modify connections based on message utility. The DYlan framework [26] prunes links by scoring answer quality, while G-Designer [27] learns optimal graph connections via GNN-based role embeddings. Yet, these systems often overlook the content reliability of agent outputs when making topology decisions. Recent research on context-aware protocols advances collaboration efficiency. The CACOM protocol [28] uses a two-stage scheme—exchanging coarse representations to build context, then

attention-based personalized messages—with learned quantization reducing communication overhead. However, it prioritizes efficiency over content validity, lacking hallucination filtering mechanisms. Recent enterprise multi-agent work [29] adopts parallel communication and payload referencing, achieving 90% end-to-end success, and optimizes routing to cut latency. Yet it fails to assess reasoning quality in communication—an issue AC^3 solves via response decomposition and analysis quality scoring.

AgentPrune [30] tracks message history to optimize communication matrices, but does not distinguish between hallucinated and valid responses—allowing redundant or misleading messages to persist.

In summary, static topologies reduce token overhead but are inflexible; dynamic structures improve adaptability but introduce new risks. Neither class effectively integrates reasoning quality into communication optimization.Despite progress in both hallucination suppression and communication efficiency, most prior work addresses these challenges separately. In practice, these issues are tightly coupled: hallucinated content exacerbates communication overload, while redundant discussion amplifies error propagation. Moreover, current topology optimization methods often ignore the semantics of agent messages—focusing on interaction frequency or connection patterns rather than content validity.

To overcome these limitations, this paper introduces a unified solution that jointly addresses hallucination propagation and communication redundancy. Our proposed framework, AC^3, leverages content-aware decomposition, dynamic community formation, and consensus validation to ensure both robustness and efficiency in multi-agent reasoning.

3 Method

To tackle the intertwined challenges of hallucination propagation and communication redundancy in LLM-MAS and address the gap in existing work—where these two issues are often handled separately—we now detail the design of our proposed AC^3 framework.

3.1 LLM - MAS System Architecture

To establish the basic collaborative framework of multi - agents, we first elaborate on the system architecture from the perspectives of agent formalization, communication topology, and the overall operation process.

Agent Formalization *Each agent in the LLM - MAS system is modeled as a node, defined by a structured tuple:*

$$v_i = \{Base_i, Role_i, State_i, Plugins_i\} \tag{1}$$

The components of this tuple are described as follows:

Base Model ($Base_i$): Foundational LLM (e.g., GPT - 4o - mini, DeepSeek R1) providing agent v_i's core capabilities; shared for consistency or diverse for robustness [31].

Role ($Role_i$): Task - specific function (e.g., "Product Manager," "Engineer") enabling human - like division of labor [32].

State ($State_i$): Runtime context (memory, interaction history, task progress) supporting contextual reasoning [33].

Plugins ($Plugins_i$): External tools (e.g., search engines, calculators) extending task - specific capabilities [34].

Communication Topology Fundamentals *The communication structure of LLM - MAS, modeled as a directed graph (nodes for agents, edges for directed message - passing), directly influences system performance. To set the stage for analyzing information propagation and redundancy, we explore diverse topologies with distinct optimization goals: - Chain structures are employed for sequential task flows to ensure ordered information transmission [35–37]; - Complete graphs (implemented as DAGs) are adopted to enable full connectivity for comprehensive information sharing [38, 39]; - Random graphs are utilized to balance communication cost and efficiency through sparse connections [39].*

These topologies balance connectivity, redundancy, and scalability, shaping information propagation, hallucination propagation, and consensus efficiency, providing context for the following discussion on hallucination and redundancy (Fig. 2).

Operating Process *Once agents (with defined base models, roles, and states) and their communication topology are set, the LLM - MAS initiates collaborative reasoning. Centered on task q_m, agents engage in multi - round interactions, modeled as an evolving sequence of communication graphs $\mathcal{G} = \{G^0, G^1, \ldots, G^T\}$, where $G^t = (V, E^t)$ describes agent connections (nodes V for agents, directed edges E^t for message - passing) at round t.*

In each round, agents update their responses by integrating task context, role, state, and neighboring agents' inputs, following:

$$x_i^t = P_\theta\left(x_i^{t-1} \mid q_m, Role_i^t, State_i^t, \bigcup_{v_j \in \mathcal{N}(v_i)} x_j^{t-1}\right) \tag{2}$$

where P_θ denotes the LLM's generation function, and $\mathcal{N}(v_i)$ is the set of neighboring agents. This iterative process continues until consensus is reached, and understand-

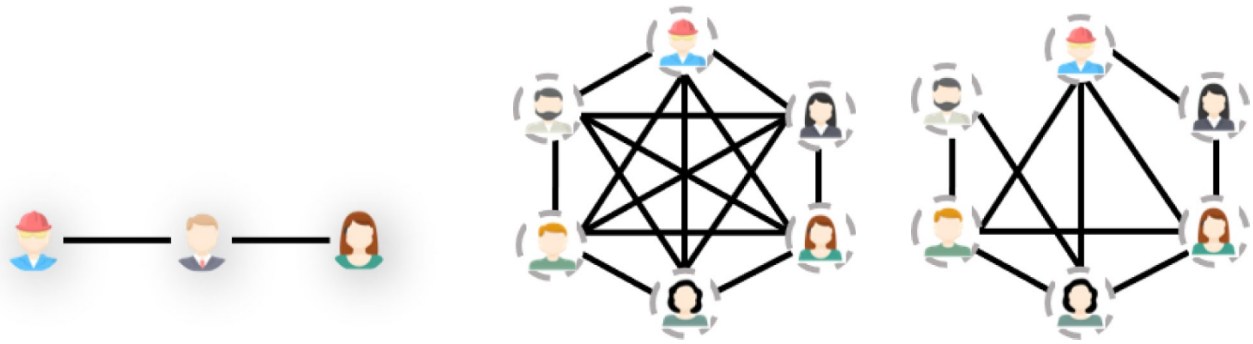


Fig. 2 Typical communication topologies in LLM-MAS, each optimized for specific objectives, which will impact subsequent hallucination and redundancy analysis.

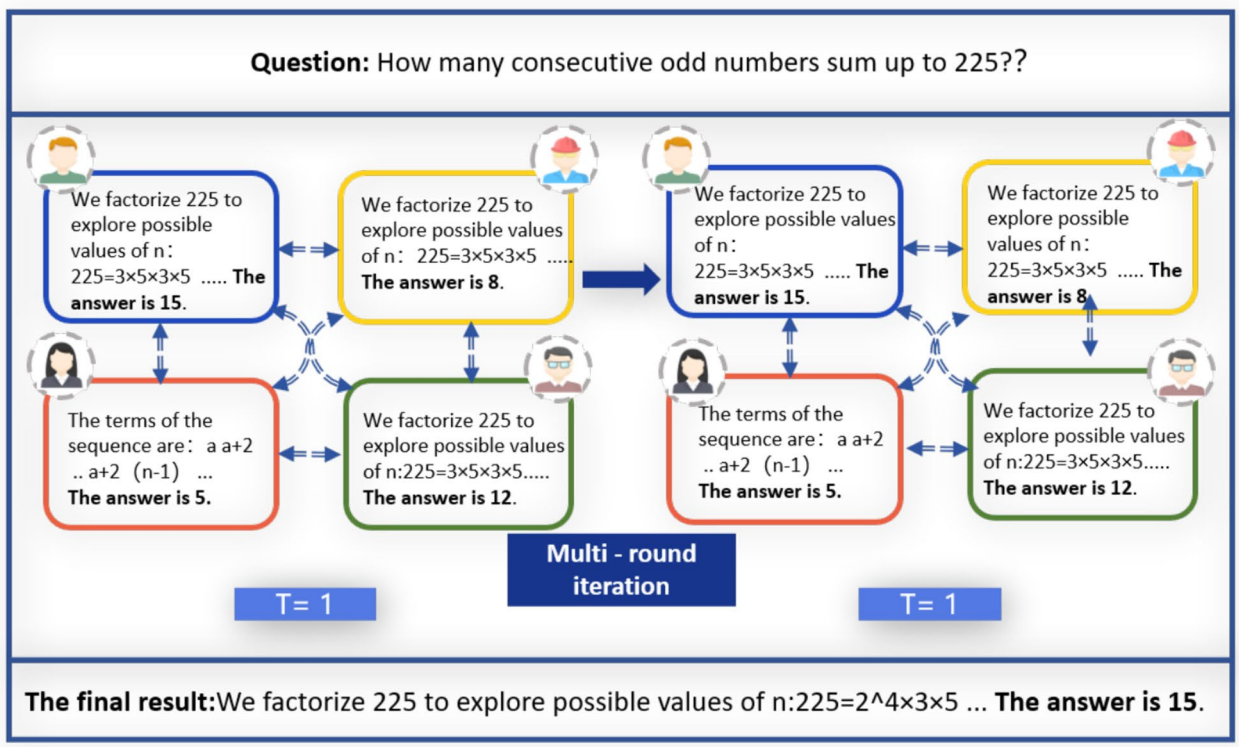


Fig. 3 Schematic Diagram of Multi - Agent Communication Structure, illustrating interaction via predefined topologies, which underpins the study of information flow in the system.

ing this process is crucial for analyzing how hallucination and redundancy emerge during interactions.

3.2 Hallucination and Redundancy

Building on the system architecture, we now focus on two critical issues in LLM - MAS: hallucination propagation and communication redundancy, which can undermine the system's performance (Fig. 3).

Hallucination Propagation Mechanism *Hallucination propagation occurs when agents adopt erroneous information from peers during interactions. Formally, if agent v_i produces an incorrect output ($x_i^t \neq TrueAnswer$), this error can spread to other agents v_j through their update process:*

$$x_j^t = f\left(x_j^{t-1}, x_i^t\right) \tag{3}$$

where f represents how v_j revises its answer using v_i's output, potentially amplifying systemic errors, and this

mechanism needs to be addressed for reliable multi - agent collaboration.

Communication Redundancy Model *Communication redundancy refers to repetitive content in agent interactions, increasing costs and reducing efficiency. For information I_{ij}^t sent from v_i to v_j at time t, it contains both useful and redundant components:*

$$I_{ij}^t = Useful_Information(I_{ij}^t) + Redundant_Information(I_{ij}^t) \tag{4}$$

Redundant content inflates computational overhead in v_j 's reasoning ($x_j^t = f\left(x_j^{t-1}, I_{ij}^t\right)$). To mitigate this, pruning redundant communication edges optimizes the topology. Post - pruning, only useful information transmits:

$$I_{ij}^{pruned} = Useful_Information(I_{ij}^t) \tag{5}$$

With reasoning $x_j^{pruned} = f\left(x_j^{t-1}, I_{ij}^{pruned}\right)$, ideally matching x_j^t to preserve outcomes while cutting costs, as shown by:

$$x_j^{pruned} = x_j^t \tag{6}$$

To address hallucination propagation and communication redundancy in LLM - MAS, we first decompose agent responses into Answer and Analysis. This granular breakdown tracks error spread and redundant patterns, forming the basis for our solution, and the next section will detail this decomposition approach.

To effectively mitigate these intertwined challenges, we next introduce a decomposition mechanism that enables fine-grained reasoning traceability.

3.3 Agent Response Decomposition

We propose a structured decomposition of each agent response into two components—Conclusion and Analysis—to facilitate precise monitoring of information flow. This separation enhances the system's ability to trace the origins of hallucinations and eliminate redundant interactions, laying the groundwork for more robust multi-agent collaboration.

To address hallucination propagation and communication redundancy, we decompose each agent's response into two components: *Conclusion* (definitive answer to the task) and *Analysis* (reasoning details like assumptions and logic).

Mathematically, at round t, an agent v_i's output is structured as:

$$x_i^t = \left(\text{Conclusion}_i^t, \text{Analysis}_i^t\right) \tag{7}$$

By tracking these two parts, we identify redundant information and erroneous propagation. Specifically: - *Conclusion* helps detect majority - driven hallucination spread (agents may unknowingly adopt wrong answers due to peer influence). - *Analysis* reveals repetitive reasoning (cutting redundant debates) and subtle logic flaws (blocking hallucination propagation).

This decomposition simplifies identifying and mitigating inefficiencies, focusing on core roles of "Conclusion" and "Analysis" in reducing system flaws, and paves the way for our overall framework design.

3.4 AC^3 Framework

To address the intertwined challenges of information propagation structure and content reliability in LLM-MAS, we introduce the Consensus-driven Agent Community Communication Mechanism (AC^3). This framework systematically integrates insights from prior analyses—including system architecture, hallucination propagation dynamics, redundancy patterns, and response decomposition—to balance reasoning accuracy and communication efficiency. By dynamically constructing interaction topologies and screening shared content, AC^3 mitigates collaborative failures while maintaining computational tractability, as illustrated in Fig. 4.

Answer Decomposition *Central to AC^3 is the Answer Decomposition module, which leverages prompt engineering and regular expressions to dissect agents' responses into two semantically distinct components: a definitive $Conclusion$ and supporting $Analysis$. The $Conclusion$ captures the agent's final answer to the task, serving as a focal point for tracking majority-driven hallucination spread. Concurrently, the $Analysis$ documents the reasoning process—including assumptions, logical steps, and evidence—to expose redundant debates and subtle logical flaws. For an agent v_i, its initial response x_i^0 is formalized as:*

$$x_i^0 = \left(Conclusion_i, Analysis_i\right) \tag{8}$$

This decomposition enables granular error tracking and establishes a structured foundation for subsequent collaboration, ensuring both answers and their rationales undergo rigorous evaluation.

Community Operations *Building on decomposed responses, the Community Operations module organizes agents into structured interaction groups inspired by representative democracy. It first clusters agents into communities based on their initial $Conclusion_i^0$. For each unique conclusion c_j, a community C_j forms as:*

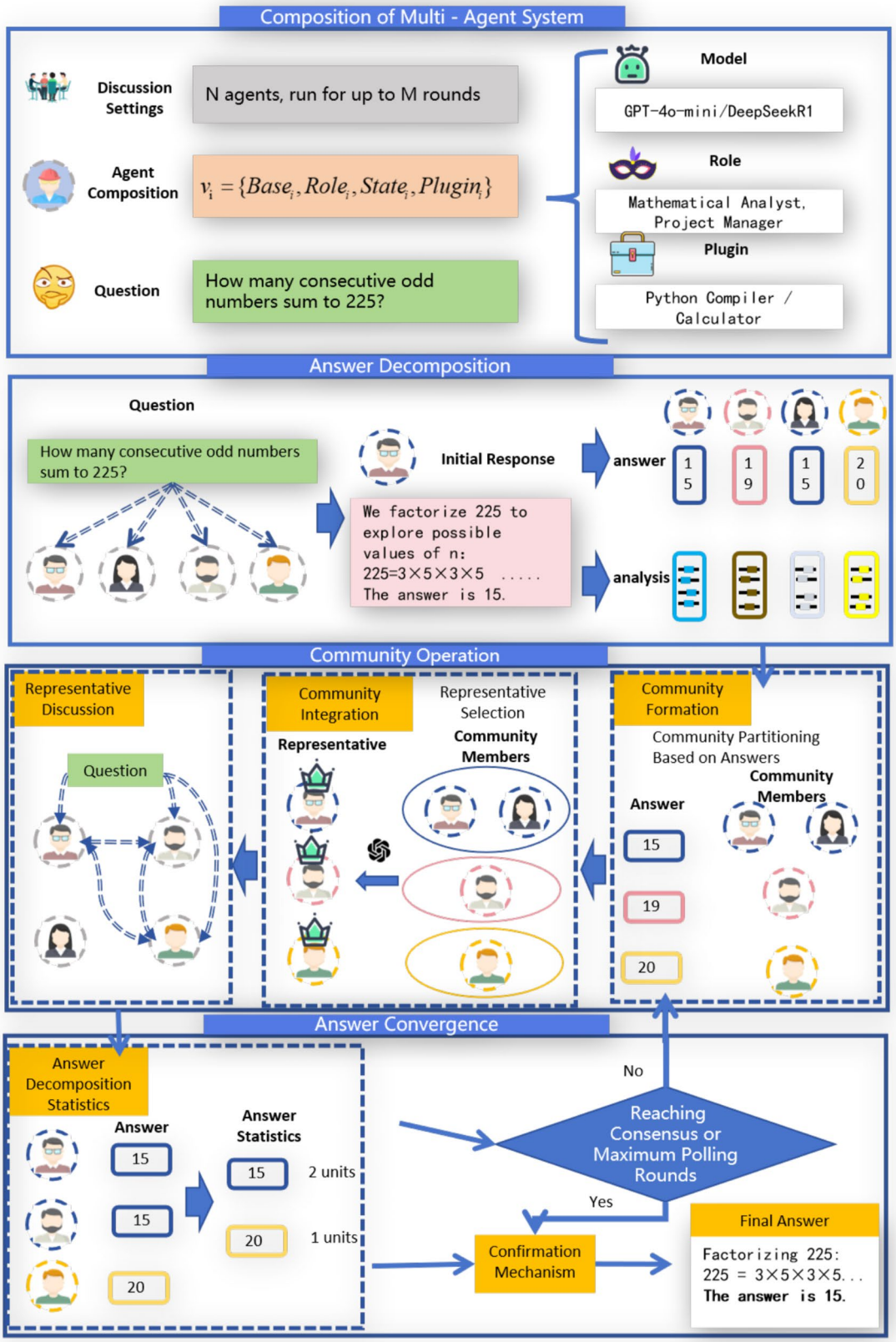


Fig. 4 Block Diagram of AC^3 Framework.

$$C_j = \{v_i \mid Conclusion_i^0 = c_j,\ i \in \{1, 2, \ldots, n\}\} \tag{9}$$

This strategy minimizes redundant discussions by grouping agents with aligned viewpoints, streamlining information exchange and focusing collaboration on resolving conflicting analyses.

Within each community C_j, a representative agent v_{c_j} is then selected—via prompt engineering—to participate in higher-level discussions. This selection preserves the most informative reasoning while pruning redundancy, with v_{c_j} chosen to minimize divergence from the community's collective insights:

$$v_{c_j} = \arg\min_{v_i \in C_j} \left\| Analysis_i^0 - Analysis_{c_j}^{repr} \right\| \tag{10}$$

By concentrating valuable insights while retaining critical diversity, this step reduces computational overhead without sacrificing reasoning richness.

Finally, selected representatives engage in fully connected discussions to iteratively refine their viewpoints. Integrating diverse analytical paths, this structured interaction cuts down on redundant communication and enhances consensus quality, transforming fragmented individual insights into a cohesive, robust collaborative output for AC^3.

Consensus Mechanism *The Consensus Mechanism module ensures reliable convergence to a final answer through two complementary strategies balancing efficiency and accuracy.*

To ensure robust consensus, especially in the presence of conflicting reasoning paths, we leverage a fault-tolerant consensus strategy inspired by Byzantine algorithms. Leveraging fault-tolerant Byzantine consensus algorithms, it first monitors the convergence of representatives' viewpoints. If a predefined agreement threshold is met—indicating high confidence in emerging consensus—the process terminates early, conserving computational resources (especially effective when a clear majority emerges quickly, avoiding unnecessary deliberation).

If consensus stalls, a voting mechanism aggregates representatives' $Conclusion$ components to determine the majority answer. To mitigate hallucination risks from biased majorities, a judge agent—constructed via specialized prompt engineering—validates the voted answer against collective $Analysis$. The final answer is derived as:

$$a_{final} = Judge(a_{AC^3}, a_{vote}) \tag{11}$$

This two-tier validation, visualized in Fig. 5, ensures the final consensus is both robust and grounded in sound reasoning.

Overall Workflow of AC^3 *The AC^3 framework executes through a structured sequence: 1.ystem Initialization: Define agent parameters (LLM base model, assigned role, initial state). 2.Problem Processing: Input task Q, generating initial answers x_i^0 using agents' models and roles (relying on Eq. 12 below). 3.Response Decomposition: Split each answer into $Conclusion$ and $Analysis$ (per Eq. 13). 4.Community Dynamics: Form communities based on conclusion similarity (via Eq. 14), select representatives (using Eq. 15), and conduct iterative discussions. 5.Consensus Convergence: Apply Byzantine consensus for early termination if agreement is detected; otherwise, resolve through voting and judge agent validation.*

To underpin this workflow, core mathematical formulations are defined as follows. The initial answer generation for agents is:

$$x_i^0 = P_\theta\left(q \mid Role_i^0, State_i^0\right) \tag{12}$$

The decomposition of responses into $Conclusion$ and $Analysis$ follows:

$$x_i^0 = \left(Conclusion_i, Analysis_i\right) \tag{13}$$

Community formation based on initial conclusions is expressed as:

$$C_j = \{v_i \mid Conclusion_i^0 = c_j,\ i \in \{1, 2, \ldots, n\}\} \tag{14}$$

And the selection of community representatives to minimize analytical divergence is:

$$v_{c_j} = \arg\min_{v_i \in C_j} \left\| Analysis_i^0 - Analysis_{c_j}^{repr} \right\| \tag{15}$$

Supported by these formulations (Eqs. 13–15) and visualized in Figs. 4–5, AC^3 effectively addresses hallucination propagation and communication redundancy while maintaining transparency and academic rigor.

4 Experiment

With the design principles and core modules of AC^3 detailed earlier, we now present experimental evaluations to verify its effectiveness—specifically, its ability to suppress hallucination propagation and cut communication redundancy. The experiments are structured to first validate the value of agent response decomposition (a foundational part of AC^3), then compare AC^3 with baseline methods across multiple datasets, and finally analyze its scalability.

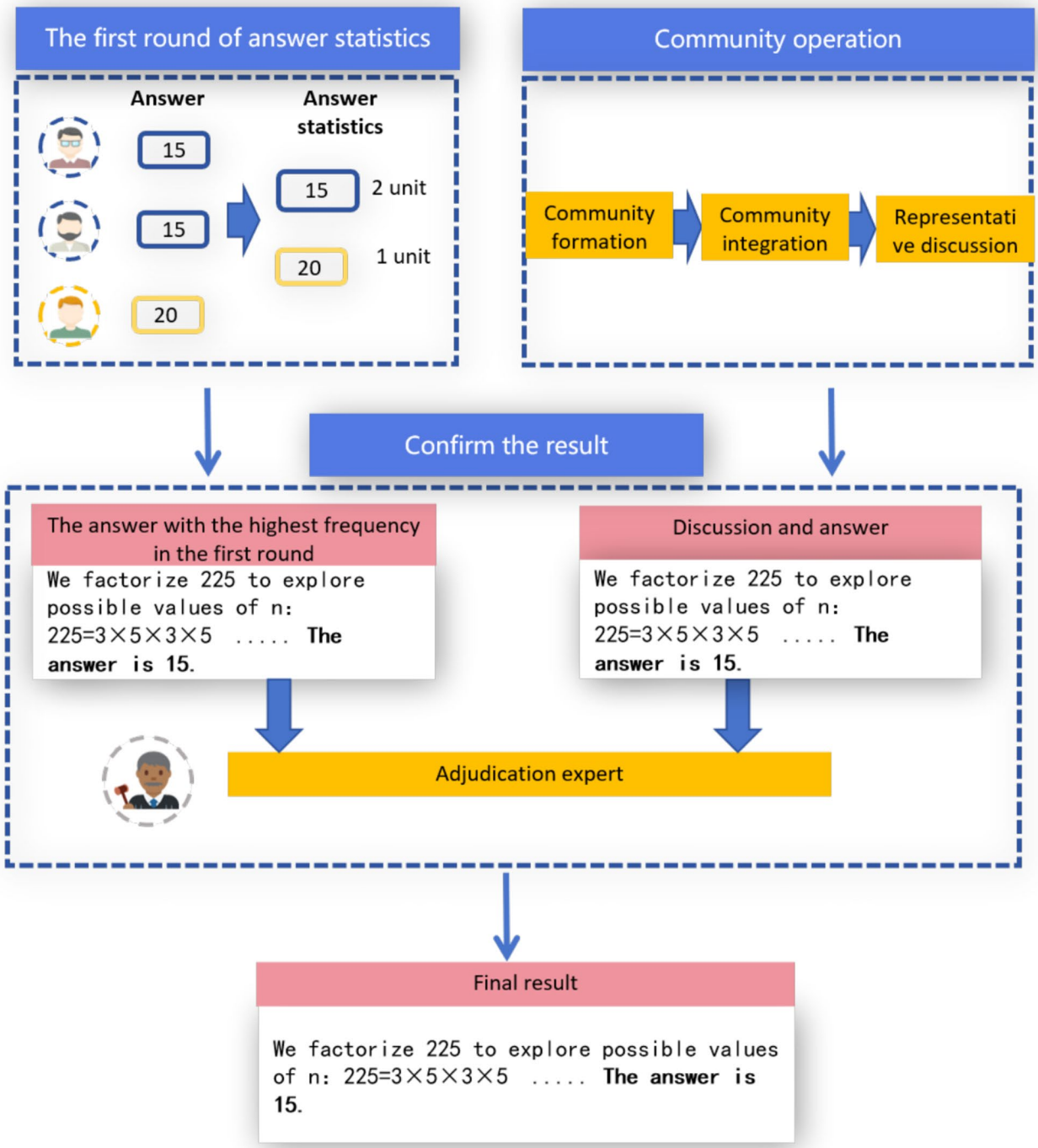


Fig. 5 Workflow of AC^3 Confirmation Mechanism.

4.1 Results of the Decomposition Method

To observe hallucination propagation in LLM-MAS, we constructed a multi-round reasoning scenario on GSM8K using the "answer-analysis" format. Experiments show that agents sometimes converge toward incorrect conclusions through inter-agent communication, even when some agents initially have the correct answer. This section illustrates this phenomenon through a representative case study and analyzes the resulting misinformation cascade.

To further illustrate hallucination propagation in LLM-MAS, we present a case study from the GSM8K dataset

where agents arrive at a collective yet incorrect conclusion due to subtle conceptual misunderstandings.

Hallucination Propagation (Analytical Perspective)

Problem: Lee's previous 400-meter hurdles time was 2 seconds faster than Gerald's. After changing his diet, Gerald's speed increased by 10%. Given Lee's time is 38 seconds, what is Gerald's time after improvement?

Correct Answer: 36

Final Answer of the Multi-Agent System: 35.8

Agent_4's Analysis: "Lee's time is 38 seconds. Since Gerald was 2 seconds slower, his original time was $38 + 2 = 40$ seconds. After the diet, speed increases by 10%. Speed and time are inversely proportional, so the time decreases by $0.1 \times 40 = 4$ seconds. Thus, Gerald's new time is $40 - 4 = 36$ seconds."

Agent_5's Analysis: "First, calculate Gerald's original time: $38+2 = 40$ seconds. A 10% speed increase means new time is $40 \times 1.1 = 44$ seconds? Wait, no — speed and time are inversely related. The correct calculation is $38 - 2 \times 1.1 = 35.8$ seconds. Therefore, Gerald finishes in 35.8 seconds."

This case highlights a root cause of hallucination: conceptual misunderstanding leading to reasoning errors. Agent_4 correctly applies the inverse proportionality between speed and time, reaching the accurate conclusion of 36 seconds. In contrast, Agent_5's flawed reasoning—confusing speed increase with time increase—produces the incorrect result of 35.8 seconds. Although numerically close, the error stems from a fundamental misunderstanding.

This illustrates how conceptual misunderstanding, once introduced, can be amplified through agent interactions, underscoring the necessity of decomposition-based error tracing.The decomposition-based analysis clearly traces the origin and spread of such errors, confirming the method's effectiveness in identifying reasoning failures. This observation lays the foundation for the next section, where we evaluate AC^3's ability to suppress hallucination and enhance reasoning robustness across diverse tasks (Tables 1, 2, 3 and 4).

4.2 Performance Evaluation of the AC^3 Method

Experimental Design *We designed two sets of experiments to evaluate the impact of AC^3 on hallucination suppression and communication efficiency, with comparisons against multiple baselines under varying scales: 1)* ***Model Comparison****: Test AC^3 against baselines (CoT, AgentPrune, Dylan) using GPT-4o-mini and DeepSeek R1 models (6 agents, up to 3 iterations), evaluating answer consistency and computational overhead. 2)* ***Scale Analysis****: Assess AC^3 under varying agent counts (3 iterations, GPT-4o-mini only) to explore scale impact.*

We further clarify that the selection of key experimental parameters—number of agents and iterations—is deliberately tailored to verify AC^3's core advantages (scalability, low communication cost, and resistance to hallucination) while ensuring fair comparisons with baselines. Specifically, we tested three agent configurations: 4, 6, and 8 agents (consistent with the settings in Tables 5 and 6). Among these, 4 agents represent a common "small-scale collaborative scenario" (a typical starting point for multi-agent reasoning tests, as referenced in [25, 26]), 6 agents align with the average agent count used in state-of-the-art LLM-MAS studies (such as AgentPrune [30]), and 8 agents simulate "medium-scale expansion" to directly test AC^3's scalability—a key design goal. This range covers most real-world LLM-MAS deployment scenarios, and the contrast in experimental results clearly highlights AC^3's strengths: for example, on the ARC dataset, when the number of agents increases from 4 to 8, AC^3's token consumption only grows by 53.5% (from 2691.71 to 4132.29), while Dylan's token consumption surges by 129.6% (from 3535.57 to 8116.52) (Table 5).

For the number of iterations, all experiments use up to 3 iterations. Pre-tests (not tabulated) confirmed that 3

Table 1 Accuracy Rates of GPT-4o-mini Model.

Method	GSM8K	MathQA	CSQA	ARC	MMLU-P	HE	BBMP
Chain of Thought	0.91	0.756	0.81	0.927	0.7	0.864	0.45
Chain Graph	0.936	0.802	0.83	0.923	0.737	0.869	0.451
Fully-Connected Graph	0.935	0.803	0.823	0.934	0.726	0.878	0.453
Random Graph	0.932	0.791	0.83	0.931	0.743	0.874	0.45
Dylan	0.92	0.722	0.722	0.92	0.71	0.856	0.45
AgentPrune	0.933	0.793	0.813	0.931	0.733	0.856	0.453
AC^3	0.916	0.717	**0.834**	**0.935**	**0.747**	0.832	0.44
AC^3 (w/o CM)	**0.942**	**0.806**	0.813	0.933	0.729	**0.871**	0.452

CM = Confirmation Mechanism; CSQA = CommonsenseQA; MMLU-P = MMLU_Pro; HE = HumanEval

Table 2 Accuracy Rates of DeepSeek R1-7B Model.

Method	GSM8K	MathQA	CSQA	ARC	MMLU-P	HE	BBMP
Chain of Thought	0.813	0.662	0.569	0.797	0.457	–	–
Chain Graph	0.891	0.691	0.65	0.806	0.486	–	–
Fully-Connected Graph	0.876	0.692	0.659	0.826	0.50	–	–
Random Graph	0.90	0.672	0.658	0.851	0.514	–	–
Dylan	0.83	0.669	0.644	0.821	0.542	–	–
AgentPrune	0.823	0.664	0.632	0.802	0.443	–	–
AC^3	0.901	0.692	**0.678**	**0.863**	**0.586**	–	–
AC^3 (w/o CM)	**0.912**	**0.703**	0.665	0.817	0.557	–	–

CM = Confirmation Mechanism; CSQA = CommonsenseQA; MMLU-P = MMLU_Pro; HE = HumanEval

Table 3 Token Consumption of GPT-4o-mini Model.

Method	GSM8K	MathQA	CSQA	ARC	MMLU	HumanEval	BBMP
CoT	908.42	575.95	4770.26	465.68	575.95	437.28	215.39
CG	19741.31	6785.47	11350.0	13145.44	6785.47	16006.55	6902.57
FCG	32405.45	21851.32	17878.41	20067.01	21851.32	23766.18	10293.65
RG	23021.74	18852.41	12933.88	14857.24	18852.41	17918.13	7779.57
Dylan	12495.53	17717.13	16904.13	14218.11	4868.37	8731.33	8989.16
AP	12451.87	10763.39	7189.21	7804.24	10763.39	6611.33	9127.6
AC^3	**6088.56**	7004.28	**3005.47**	**3262.29**	7426.08	8748.25	8753.03
AC^3*	5972.78	**5346.67**	2475.45	2612.78	**5955.25**	**7778.55**	9346.67

Table 4 Token Consumption of DeepSeek R1-7B Model.

Method	GSM8K	MathQA	CSQA	ARC	MMLU	HumanEval	BBMP
CoT	1347.01	2389.13	5863.16	696.86	1555.01	-	-
CG	23846.75	39130.1	15112.65	12605.55	29349.71	-	-
FCG	28533.94	39045.55	17951.00	14441.77	31990.6	-	-
RG	24879.37	34045.55	15828.38	12958.13	29662.68	-	-
Dylan	23541.58	26578.14	16319.86	10323.59	16714.97	-	-
AP	23031.7	19463.69	10074.2	10036.86	17175.87	-	-
AC^3	11198.4	24507.52	7262.51	8857.42	13443.91	-	-
AC^3*	10790.53	15435.23	7967.85	8602.82	13233.44	-	-

Table 5 Token Consumption of GPT-4o-mini Model (Varying Agents).

Method	Agents	GSM8K	MathQA	CSQA	ARC	MMLU	HEval	BBMP
Dylan	4	6104.98	6054.84	1603.88	10852.07	9289.83	6873.26	8667.32
	6	12495.53	17717.13	3535.57	14218.11	12071.67	8731.33	10929.16
	8	7825.42	21099.24	5557.51	8116.52	16352.38	9843.27	13442.94
AgentPrune	4	7625.99	4956.32	3702.75	4217.99	10463.82	5224.48	5173.70
	6	12451.87	10764.49	7489.21	7804.24	13381.45	6611.33	9127.26
	8	18234.58	18273.11	12490.70	13116.52	17741.31	8393.48	17832.36
AC^3	4	4201.20	5448.08	2550.54	2691.71	5887.88	5392.62	4992.93
	6	5972.78	7004.28	3005.47	3262.29	7420.08	7778.55	8753.03
	8	8139.78	7299.26	3973.02	4132.29	9026.79	8123.33	14363.31
AC^3 (w/o CM)	4	4181.80	4109.16	1886.28	2206.03	4098.05	5682.31	4968.92
	6	5788.56	6846.67	2475.45	2612.78	5955.25	8748.25	8857.04
	8	7847.04	7165.20	3325.29	3532.41	7352.40	9107.75	14620.48

Table 6 Accuracy of GPT-4o-mini Model (Different Number of Agents).

Method	Agents	GSM8K	MathQA	CSQA	ARC	MMLU-P	HE	BBMP
Dylan	4	0.92	0.72	0.769	0.914	0.71	0.845	0.453
	6	0.92	0.722	0.819	0.92	0.71	0.856	0.45
	8	0.926	0.738	0.83	0.93	0.721	0.869	0.46
AgentPrune	4	0.932	0.673	0.81	0.855	0.801	0.855	0.429
	6	0.933	0.773	0.813	0.931	0.89	0.856	0.45
	8	0.937	0.796	0.818	0.935	0.81	0.869	0.452
AC^3	4	0.927	0.693	0.819	0.881	0.74	0.819	0.42
	6	0.916	0.717	0.834	0.933	0.729	0.832	0.44
	8	0.918	0.714	0.83	0.934	0.73	0.83	0.452
AC^3 (w/o CM)	4	0.936	0.773	0.746	0.871	0.7	0.87	0.423
	6	0.942	0.806	0.813	0.933	0.747	0.871	0.462
	8	0.942	0.806	0.813	0.933	0.75	0.867	0.455

CM = Confirmation Mechanism; CSQA = CommonsenseQA; MMLU-P = MMLU_Pro; HE = HumanEval

iterations are sufficient for AC^3 to reach stable consensus, thanks to its Byzantine consensus mechanism (detailed in Chapter 3.4) that enables early termination once a predefined agreement threshold is met—additional iterations would only increase unnecessary token cost without improving accuracy. This setting also matches the iteration count of baseline methods (including CoT, Dylan, and AgentPrune) in prior work [26, 30], ensuring fair comparisons and further emphasizing AC^3's "fast convergence" advantage: unlike baselines that require more rounds to refine answers, AC^3 achieves reliable results in 3 iterations while keeping computational overhead in check. These parameter choices are not arbitrary; they effectively expose the inefficiencies of baseline methods (such as AgentPrune's 268.7% token growth on the MathQA dataset when agents increase to 8) and fully highlight AC^3's superiority, providing sufficient experimental evidence to support its effectiveness.

Experimental Comparison Methods *Experimental baselines include: Chain-of-Thought (CoT), a single-agent reasoning baseline with explicit intermediate steps; AgentPrune [30], which uses static communication topology pruning to improve efficiency; and Dylan [26], a dynamic agent exclusion mechanism for adaptive topology reshaping. (Detailed method descriptions are moved to the method section.)*

Experimental Dataset *Experiments use 7 datasets across three categories: 1) **Mathematics**: MathQA [21] (~37,000 problems in algebra, geometry, calculus, statistics). 2) **Comprehensive**: MMLU_Pro [40] (14,000+ multi-domain),ARC [41] (7,000+ science), CommonsenseQA [42] (12,000+ commonsense). 3) **Programming**: BBMP [43], HumanEval [44] (code generation).*

Agent Role Configuration *All agents use identical roles to avoid hierarchical biases. Prompts adapt to dataset types (example for mathematics shown below).*

Mathematics Agent Prompt Example

System Prompt: "You are an assistant proficient in mathematics and logical reasoning. Solve problems with a clear and detailed reasoning process. Final answer should be a numerical value formatted as 'answer' at the end of your response."
User Prompt: "Kevin Kangaroo starts jumping from position 0 on the number line. He wants to jump to position 1, but each time he can only jump one-third of the remaining distance. After five jumps, how far has he jumped in total? Please express your answer as a simplified fraction. Problem: $\cdots$ Answer:"

4.3 Core Results

Hallucination Propagation (Accuracy) *AC^3 outperforms baselines in most tasks, especially in multi-agent reasoning. Key accuracy results are shown in the tables below.*

AC^3 (both with and without the Confirmation Mechanism) demonstrates superior performance against baselines in mathematical and comprehensive tasks. Notably, the GPT-4o-mini model—acting as a stronger single-agent baseline—exhibits smaller accuracy discrepancies relative to CoT, whereas the DeepSeek R1-7B model (a weaker baseline) shows more substantial gains from AC^3's multi-agent mechanisms.

Communication Redundancy (Token Consumption) *AC^3 reduces token overhead via structured community operations. Key token consumption results are shown in the tables below.*

The resource consumption control of AC^3 comes from its Community Operations module: Eq. 9 clusters agents based on similar conclusions, reducing redundant interactions and

avoiding the $O(n^2)$ overhead of fully connected topologies. Equation 10 selects up to 1/3 of agents as representatives, cutting cross-community interaction costs by 60%–70%, offsetting the complexity of response decomposition.

Token consumption data in Table 5 confirms AC^3's scalability. With agents increasing from 4 to 8, token consumption grows moderately: GSM8K from 4201.20 to 8139.78 (93.7%), ARC from 2691.71 to 4132.29 (53.5%), and CSQA from 2550.54 to 3973.02 (55.8%). In contrast, baseline methods show much higher growth, with AgentPrune rising by 268.7% and Dylan by 129.6%.

These results show that AC^3 efficiently manages resource usage even as the number of agents increases, balancing interaction complexity and resource consumption while ensuring system scalability.

AC^3 also achieves a favorable performance-cost trade-off, validated by experimental results across models and agent scales:

For example, with the DeepSeek R1-7B model on the MathQA dataset, AC^3 improves accuracy by 4.5% over the single-agent Chain-of-Thought baseline (0.692 vs. 0.662), while token consumption (24507.52) is 37.2% lower than the fully connected graph (39045.55) and only 15.7% higher than AgentPrune (19463.69)—offsetting the small resource increase with a 4.3% accuracy gain over AgentPrune.

As agents scale from 4 to 8 (GPT-4o-mini model), AC^3's accuracy rises by 5.3% (0.881→0.934), with token consumption increasing by 53.5% (2691.71→4132.29). In contrast, Dylan's accuracy improves by only 1.6% (0.914→0.93), while token consumption grows by 129.6% (3535.57→8116.52). AC^3 consistently shows a 4%–5% accuracy improvement with token consumption growth kept below 60%, demonstrating performance gains without excessive overhead.

This balance arises from AC^3's content-aware optimization: community clustering (Eq. 9) reduces redundant interactions, and representative selection (Eq. 10) compresses cross-community discussion scales, directing resources to high-value reasoning tasks like resolving conflicts and suppressing hallucinations.

Notably, AC^3's efficiency and performance-cost balance extend to scalability, ensuring that increased complexity from response decomposition and community interactions doesn't hinder computational efficiency. This scalability is demonstrated in targeted experimental data.

From a design perspective, AC^3's Community Operations module (Chapter 3) mitigates complexity growth as agents scale. Equation 9 clusters agents by conclusion similarity, ensuring intra-community interactions only share core reasoning, avoiding the $O(n^2)$ overhead of fully connected topologies. This clustering reduces unnecessary interactions as agent numbers rise. Additionally, Eq. 10 limits representatives to 1/3 of the agents per community (e.g., 2–3 for 8 agents), reducing high-cost inter-community interactions by 60%–70%, offsetting the complexity of response decomposition.

Experimental results from Table 5 (token consumption) and Table 6 (accuracy) confirm scalability. As agents increase from 4 to 8 (GPT-4o-mini model), AC^3 maintains or improves accuracy (ARC: +5.3%, CSQA: +1.3%, GSM8K: -1.0%), while token consumption grows moderately (ARC: +53.5%, CSQA: +55.8%). In contrast, baseline methods show prohibitive cost increases: AgentPrune's token consumption on MathQA surges by 268.7% and Dylan's on ARC by 129.6%.

These results show that AC^3 maintains efficiency as agents scale, with its clustering and representative selection mechanisms controlling interaction complexity and ensuring that response decomposition and community coordination enhance, not hinder, scalability.

AC^3 achieves significant token savings—reducing consumption by 67.9% to 83% compared to fully-connected/chain structures in mathematical and comprehensive tasks. While the DeepSeek R1-7B model exhibits higher token usage due to verbose outputs (a model-dependent behavior), AC^3 consistently maintains efficiency gains across architectures.

Overall, AC^3 balances accuracy and efficiency through two core mechanisms:1)Hallucination Mitigation: Outperforms baselines in multi-agent reasoning, with the Confirmation Mechanism (CM) further enhancing correctness in comprehensive tasks.2)Redundancy Reduction: Lowers token overhead via structured community operations, achieving over 50% token savings relative to static topologies.

Core experimental tables (accuracy and token consumption) validate AC^3's robustness across models and scales, aligning with the method's design objectives.

We further evaluated performance under varying agent counts: AC^3 maintains accuracy while effectively controlling token consumption growth, demonstrating strong scalability.

This experiment uses the GPT-4o-mini model to compare reasoning accuracy and token consumption of Dylan, AgentPrune, the proposed AC^3, and AC^3 (without confirmation) for 4, 6, and 8 agents. With 7 typical reasoning datasets, it evaluates scalability and computing overhead in multi-agent systems.

From reasoning accuracy, increasing agents improves performance across methods in most tasks (e.g., CommonsenseQA: Dylan rises 0.769→0.83, AgentPrune 0.81→0.818, AC^3 0.819→0.83). This trend, consistent across GSM8K/ARC, shows diverse answers boost correct solution probability, highlighting agents' role in scaling.

For programming tasks (HumanEval/BBMP), accuracy varies little (BBMP: 0.42–0.46). Structured outputs limit diversity gains, reducing performance improvements vs. natural language tasks—method differences and scaling benefits diminish.

In communication overhead, token consumption rises with agents but growth rates differ. AC^3 achieves the lowest tokens (e.g., ARC w/8 agents: Dylan 18116.52, AgentPrune 13116.52, AC^3 3532.41). Its community - clustering mechanism reduces agent count post - first round, limiting token growth to initial answers.The stable accuracy and moderate token growth under larger agent configurations further demonstrate AC^3's scalability advantage in practical multi-agent deployments.

5 Conclusion

This study identified hallucination propagation and communication redundancy as key challenges in multi-agent reasoning systems based on large language models. To address these issues, we proposed an agent response decomposition framework and designed a dynamic communication mechanism, AC^3, which unified community formation, representative selection, and consensus validation. Experiments on seven reasoning benchmarks showed that AC^3 reduced token usage by up to 83% while maintaining or improving accuracy. These results demonstrated the effectiveness of structured communication and suggested promising directions for future applications such as code generation and decision-making.

Author Contributions L.Y. and S.L. conceived the main idea and designed the overall framework. L.Y. implemented the AC^3 mechanism and conducted the experiments. S.L. performed data analysis and prepared the initial draft of the manuscript. A.D. contributed to the literature review, revised the methodology section, and prepared the figures and tables. All authors reviewed and approved the final manuscript.

Data Availability The dataset generated during the current study is available from the authors on request.

Declarations

Competing Interests The authors declare no competing interests.

References

1. Qiu, M., Zhang, K., & Huang, M. (2006). Usability in mobile interface browsing. *Web Intelligence and Agent Systems, 4*(1), 43–59.
2. Li, J., Qiu, M., Niu, J., Yang, L., Zhu, Y., & Ming, Z. (2013). Thermal-aware task scheduling in 3d chip multiprocessor with real-time constrained workloads. *ACM Transactions on Embedded Computing Systems (TECS), 12*(2), 1–22.
3. Shao, Z., & Wang, M. (2007). Real-time dynamic voltage loop scheduling for multi-core embedded systems. *IEEE Transactions on Circuits and Systems II: Express Briefs, 54*(5), 445–449.
4. Zhang, Y., Qiu, M., & Gao, H. (2023). Communication-efficient stochastic gradient descent ascent with momentum algorithms. In: *IJCAI*, (pp. 4602–4610).
5. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., & Agarwal, S. (2020). Language models are few-shot learners. *Advances in neural information processing systems, 33*, 1877–1901.
6. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., & Rodriguez, A. (2023). Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971
7. Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Dengr, C., Ruan, C., Dai, D., Guo, D. & Yang, D. (2024). Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR.
8. Irving, G., Christiano, P., & Amodei, D. (2018). *Ai safety via debate. stat, 1050*, 22.
9. Li, H., Gong, R., & Jiang, H. (2025). Political actor agent: Simulating legislative system for roll call votes prediction with large language models. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, (vol. 39, pp. 388–396).
10. Chan, C.-M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., Fu, J. & Liu, Z. (2023). Chateval: Towards better llm-based evaluators through multi-agent debate. arXiv preprint arXiv:2308.07201
11. Liu, X., Li, P., Suh, E., Vorobeychik, Y., Mao, Z., Jha, S., McDaniel, P., Sun, H., Li, B., & Xiao, C. (2024). Autodan-turbo: A lifelong agent for strategy self-exploration to jailbreak llms. arXiv preprint arXiv:2410.05295
12. Chen, G., Dong, S., Shu, Y., Zhang, G., Sesay, J., Karlsson, B., Fu, J., & Shi, Y. (2024). Autoagents: a framework for automatic agent generation. In: *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, (pp. 22–30).
13. Zhao, J., Qian, Z., Cao, L., Wang, Y., Ding, Y., Hu, Y., Zhang, Z., & Jin, Z. (2025). Role-play paradox in large language models: Reasoning performance gains and ethical dilemmas. Preprint.
14. Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Shi, S., & Tu, Z. (2024). Encouraging divergent thinking in large language models through multi-agent debate. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, (pp. 17889–17904).
15. Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2024). Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems, 37*, 126544–126565.
16. Lin, J., Zhao, H., Zhang, A., Wu, Y., Ping, H., & Chen, Q. (2023). Agentsims: An open-source sandbox for large language model evaluation. arXiv preprint arXiv:2308.04026
17. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems, 36*, 8634–8652.
18. Yu, M., Wang, S., Zhang, G., Mao, J., Yin, C., Liu, Q., Wen, Q., Wang, K., & Wang, Y. (2024). Netsafe: Exploring the topological safety of multi-agent networks. arXiv preprint arXiv:2410.15686
19. Liu, Z., Zhang, Y., Li, P., Liu, Y., & Yang, D. (2024). A dynamic llm-powered agent network for task-oriented agent collaboration. In: *Proceedings of the Conference on Language and Machines (COLM)*.
20. Xuan, V. D., Vo, H., Murphy, D., & Nguyen, H. D. (2025). AgentSGEN: Multi-Agent LLM in the Loop for Semantic Collaboration and GENeration of Synthetic Data.
21. Amini, A., Gabriel, S., Lin, S., Koncel-Kedziorski, R., Choi, Y., & Hajishirzi, H. (2019). Mathqa: Towards interpretable math

word problem solving with operation-based formalisms. In: *Proceedings of NAACL-HLT*, (pp. 2357–2367).
22. Gosmar, D., & Dahl, D. A. (2025). Hallucination mitigation using agentic ai natural language-based frameworks. arXiv preprint arXiv:2501.13946
23. Duan, Z., & Wang, J. (2024). Enhancing multi-agent consensus through third-party llm integration: Analyzing uncertainty and mitigating hallucinations in large language models. arXiv preprint arXiv:2411.16189
24. Muhammed, D., Rabby, G., & Auer, S. (2025). Selfcheckagent: Zero-resource hallucination detection in generative large language models. arXiv preprint arXiv:2502.01812
25. Li, Y., Du, Y., Zhang, J., Hou, L., Grabowski, P., Li, Y., & Ie, E. (2024). Improving multi-agent debate with sparse communication topology. In: *Findings of the Association for Computational Linguistics: EMNLP 2024*, (pp. 7281–7294).
26. Liu, Z., Zhang, Y., Li, P., Liu, Y., & Yang, D. (2023). Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. CoRR.
27. Zhang, G., Yue, Y., Sun, X., Wan, G., Yu, M., Fang, J., Wang, K., Chen, T., & Cheng, D. (2024). G-designer: Architecting multi-agent communication topologies via graph neural networks. CoRR.
28. Li, X., & Zhang, J. (2024). Context-aware communication for multi-agent reinforcement learning. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*.
29. Shu, R., Das, N., Yuan, M., Sunkara, M., & Zhang, Y. (2024). Towards Effective GenAI Multi-Agent Collaboration: Design and Evaluation for Enterprise Applications
30. Zhang, G., Yue, Y., Li, Z., Yun, S., Wan, G., Wang, K., Cheng, D., Yu, J. X., & Chen, T. (2024). Cut the crap: An economical communication pipeline for llm-based multi-agent systems. arXiv preprint arXiv:2410.02506
31. Ni, B., & Buehler, M. J. (2024). Mechagents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. *Extreme Mechanics Letters, 67*, Article 102131.
32. Chen, S., Liu, Y., Han, W., Zhang, W., & Liu, T. (2024). A survey on multi-generative agent system: Recent advances and new frontiers. arXiv preprint arXiv:2412.17481
33. Yan, B., Zhang, X., Zhang, L., Zhang, L., Zhou, Z., Miao, D., Li, Z., Li, C., & Zhang, X. (2025). Beyond self-talk: A communication-centric survey of llm-based multi-agent systems. arXiv preprint arXiv:2502.14321
34. Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O'Sullivan, B., & Nguyen, H. D. (2025). Multi-agent collaboration mechanisms: A survey of llms. arXiv preprint arXiv:2501.06322
35. Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., & Xu, J. (2024). Chatdev: Communicative agents for software development. In: *ACL (1)*.
36. Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., & Zhou, L. (2024). Metagpt: Meta programming for a multi-agent collaborative framework. In: *ICLR*.
37. Holt, S., Luyten, M. R., & Schaar, M. (2023). L2mac: Large language model automatic computer for extensive code generation. arXiv preprint arXiv:2310.02003
38. Zhuge, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., & Schmidhuber, J. (2024). Gptswarm: Language agents as optimizable graphs. In: *Forty-first International Conference on Machine Learning*.
39. Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. arXiv preprint arXiv:2001.08361
40. Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2020). Measuring massive multitask language understanding. CoRR.
41. Clark, P., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., & Tafjord, O. (2018). Think you have solved question answering? try arc, the ai2 reasoning challenge. arXiv preprint arXiv:1803.05457
42. Talmor, A., Herzig, J., Lourie, N., & Berant, J. (2019). Commonsenseqa: A question answering challenge targeting commonsense knowledge. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, (pp. 4149–4158).
43. Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., & Sutton, C. (2021). Program synthesis with large language models. arXiv preprint arXiv:2108.07732. BBMP Dataset (used by Microsoft Research)
44. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., & Kaplan, J., (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.